

De la experimentación a la producción: Evaluación y Despliegue de Modelos

Semana 06 - Tecnologías Emergentes (IS046B) | FIS-UNCP

El despliegue como API cierra la línea de ensamblaje del modelo



El desafío actual: Sacar al modelo del notebook aislado.

Bloque 1: Diagnosticar la verdadera utilidad mediante métricas avanzadas.

Bloque 2: Preservar el conocimiento con un guardado estricto.

Bloque 3: Conectar el modelo con clientes web mediante HTTP.



Contexto Minero (Morococha): Solo el **2%** de las piezas presentan defectos de fisura.

La exactitud es una ilusión estadística ante clases desbalanceadas

98% Accuracy = 0% Utilidad Real

- **El modelo perezoso:** Una red que siempre predice “Pieza Sana” sin mirar los datos logra **98%** de exactitud.
- **Resultado industrial:** Cero defectos detectados y riesgo catastrófico.
- **Conclusión:** La exactitud general es ciega ante la minoría crítica.

La matriz de confusión aísla la naturaleza exacta del error

	Predicción: Positivo	Predicción: Negativo
Real: Positivo	**Verdadero Positivo (VP)** Acierto crítico 	**Falso Negativo (FN)** Peligro ignorado: Decir NO cuando es SÍ 
Real: Negativo	**Falso Positivo (FP)** Falsa alarma: Decir SÍ cuando es NO 	**Verdadero Negativo (VN)** Rechazo correcto 

- Desglosa el 100% de las predicciones en cuatro cuadrantes operativos.
- Diferencia entre una molestia y una catástrofe.
- Base matemática para métricas de rendimiento avanzadas.

Precisión frente a Recall: Confianza frente a Cobertura

PRECISIÓN

$$\frac{VP}{VP + FP}$$



Sistema de detección de plagas

- **Penaliza Falsos Positivos (FP).**
- ¿Cuando el modelo alerta, cuánto le creo?
- **Ejemplo agrícola**: Si la precisión es baja, el sistema emite alertas falsas. El agricultor pierde tiempo revisando plantas sanas y apaga la alarma.

RECALL (Sensibilidad)

$$\frac{VP}{VP + FN}$$



Alerta temprana de heladas

- **Penaliza Falsos Negativos (FN).**
- ¿Cuántos casos reales se nos escapan?
- Ejemplo agrícola: Si el modelo omite una helada mortal en el Valle del Mantaro, el agricultor pierde toda la cosecha. Prioridad absoluta.

El F1-Score castiga los desequilibrios extremos

$$2 \times \frac{(\text{Precisión} \times \text{Recall})}{(\text{Precisión} + \text{Recall})}$$



Única métrica consolidada cuando el negocio exige un balance.

En multiclase (ej. prendas de vestir), se calcula por categoría y se aplica un promedio macro o ponderado.

Usa la media armónica: castiga severamente si uno de los valores cae cerca de ce impidiendo trampas estadísticas.

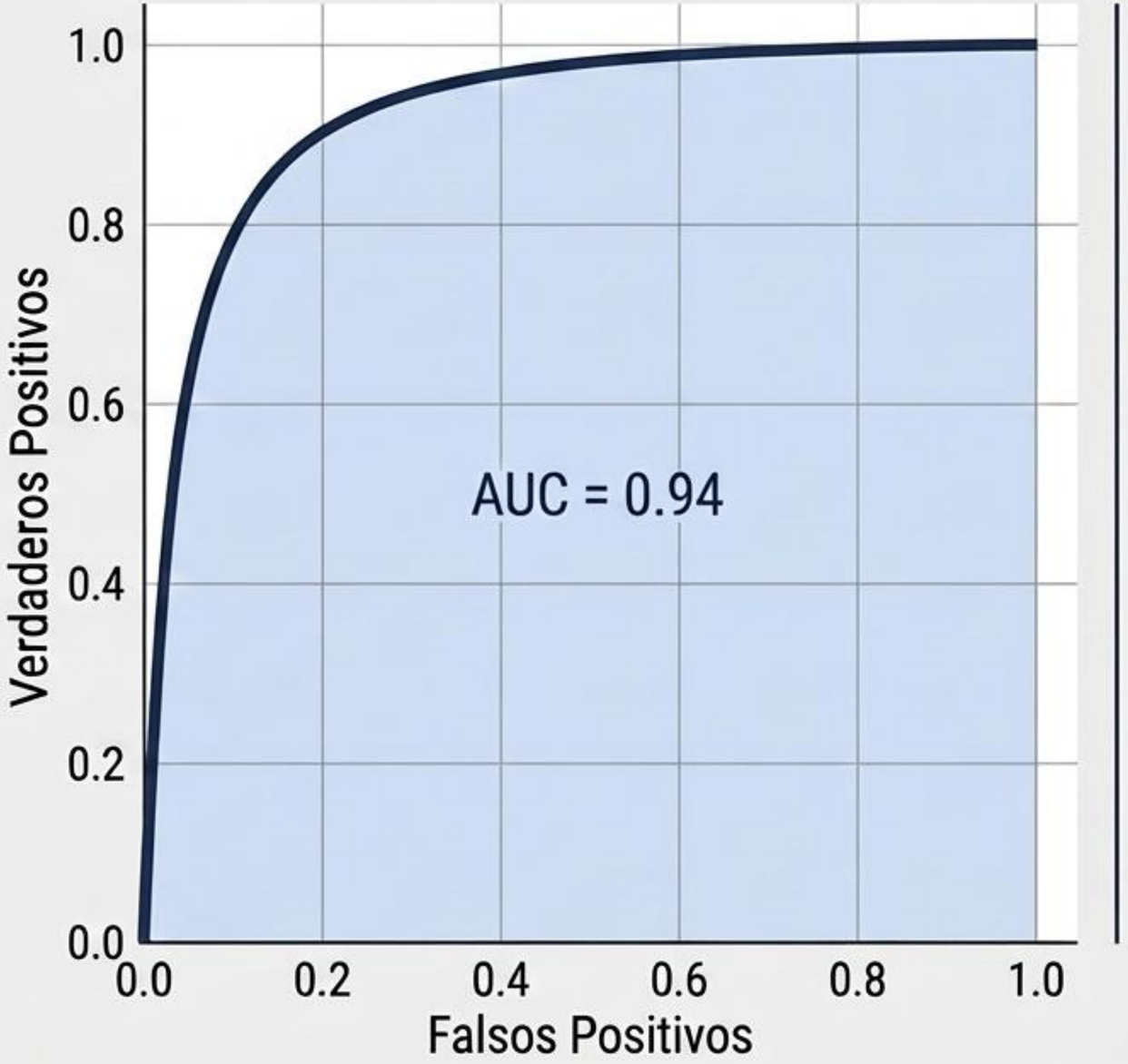
El costo financiero y humano dicta la métrica principal

Contexto del Negocio	Error de mayor impacto	Riesgo Consecuente	Métrica a Maximizar
Triaje Médico	Falso Negativo	Enviar a casa a un paciente grave	**Recall**
Fraude Bancario	Falso Negativo	Aprobar transacciones ilícitas	**Recall**
Filtro Spam	Falso Positivo	Bloquear correos urgentes de trabajo	**Precisión**
Recomendación Web	Ambos de bajo impacto	Algún clic irrelevante	**F1-Score / Accuracy**

El éxito del modelo no se define por la matemática pura, sino por el **entorno operativo**.

El ingeniero de sistemas debe **entrevistar a los expertos del dominio antes de compilar el modelo**.

Ajustar el umbral de decisión altera el modelo sin reentrenarlo

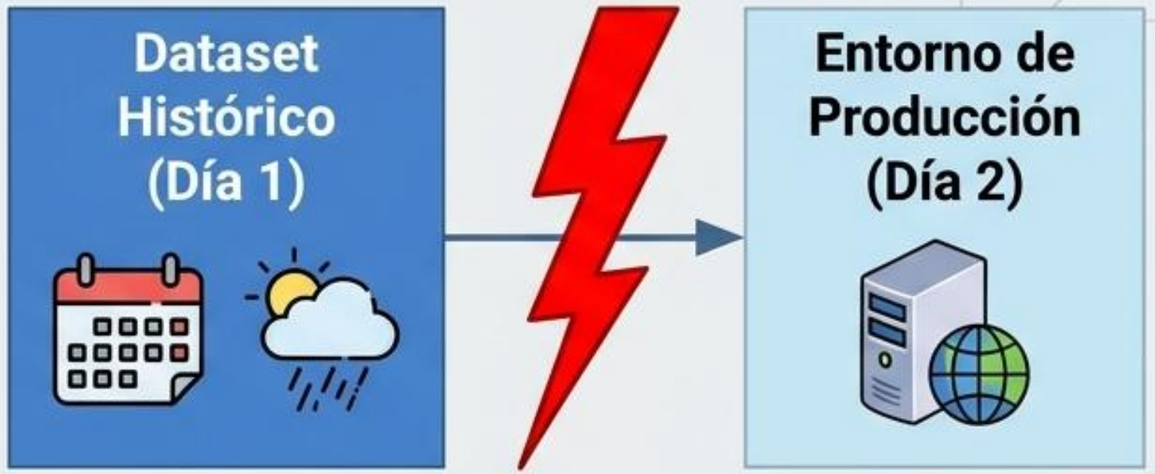




- Por defecto, el límite para predecir **SÍ** es 0.50 (50%).
- **Regla operativa**: Bajar el umbral (ej. 0.30) vuelve al modelo más paranoico. Dispara el Recall, sacrifica Precisión.
- **Curva ROC**: Visualiza visualmente todas las combinaciones posibles de este umbral.
- **AUC**: Mide la calidad absoluta (1.0 = Perfecto).

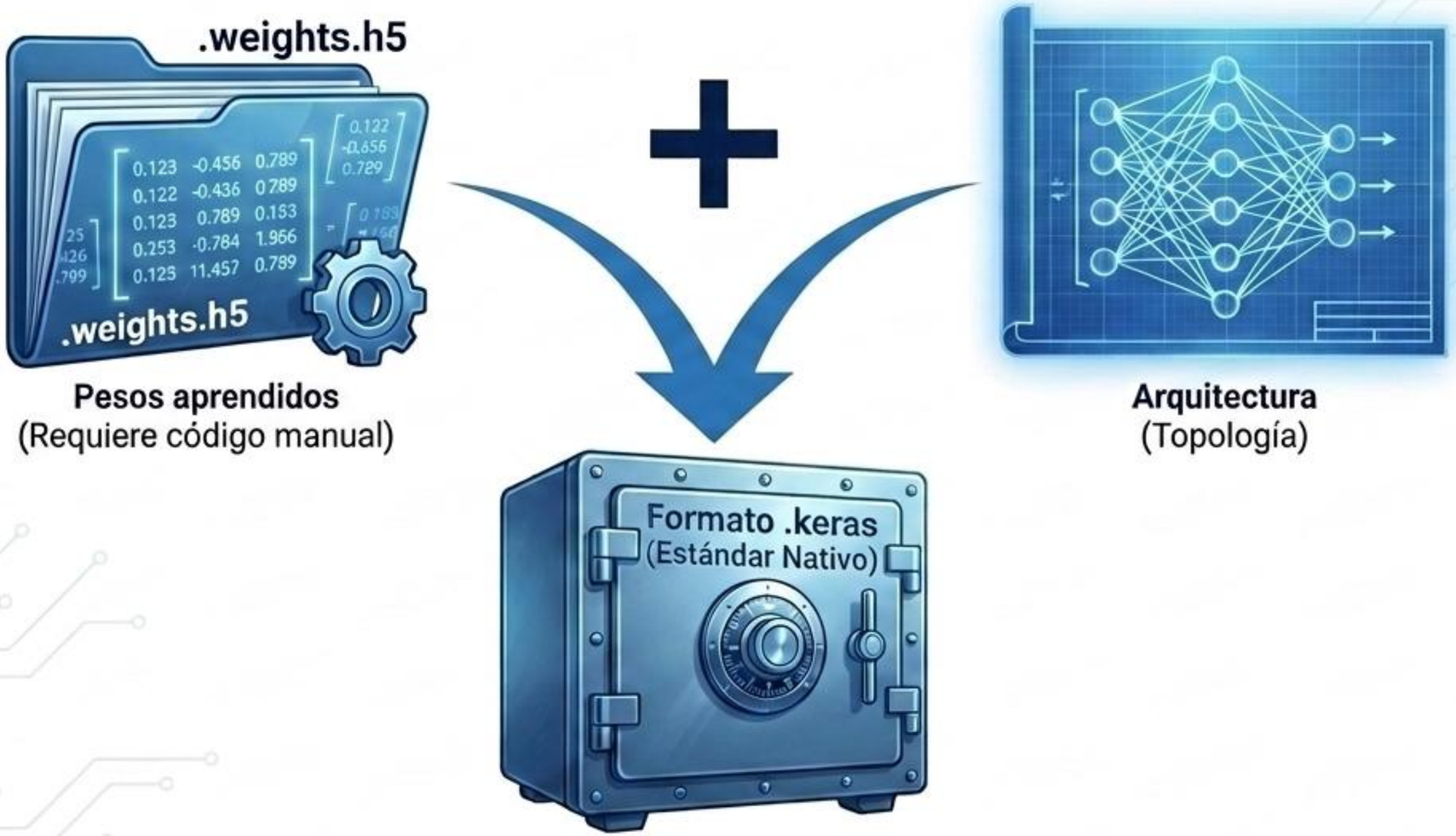
La trampa de la fuga de datos (Data Leakage)

- **Invalida** por completo la evaluación: el modelo aparenta ser perfecto (F1 = 99%).
- **Ejemplo local:** Entrenar una red para predecir si lloverá mañana en Huancayo, pero incluir por error la medición de precipitación del mismo día de mañana en las variables de entrada.
- **Resultado:** Fracaso absoluto en producción.
- **Solución:** Aislamiento estricto de variables temporales.



Datos del futuro infiltrados por error en el entrenamiento

Preservar pesos sin arquitectura es guardar una memoria sin cerebro



Pesos aprendidos
(Requiere código manual)

Arquitectura
(Topología)

Empaqueta topología, pesos y optimizador.

- **SavedModel:** Alternativa pesada para clústeres empresariales.

- **Pickle (.pkl):** Útil para scalers, inútil para Deep Learning.

- **Un modelo entrenado** representa invaluables horas de cómputo GPU. Guárdelo completo.

Versionado estricto: El DNI del modelo en producción

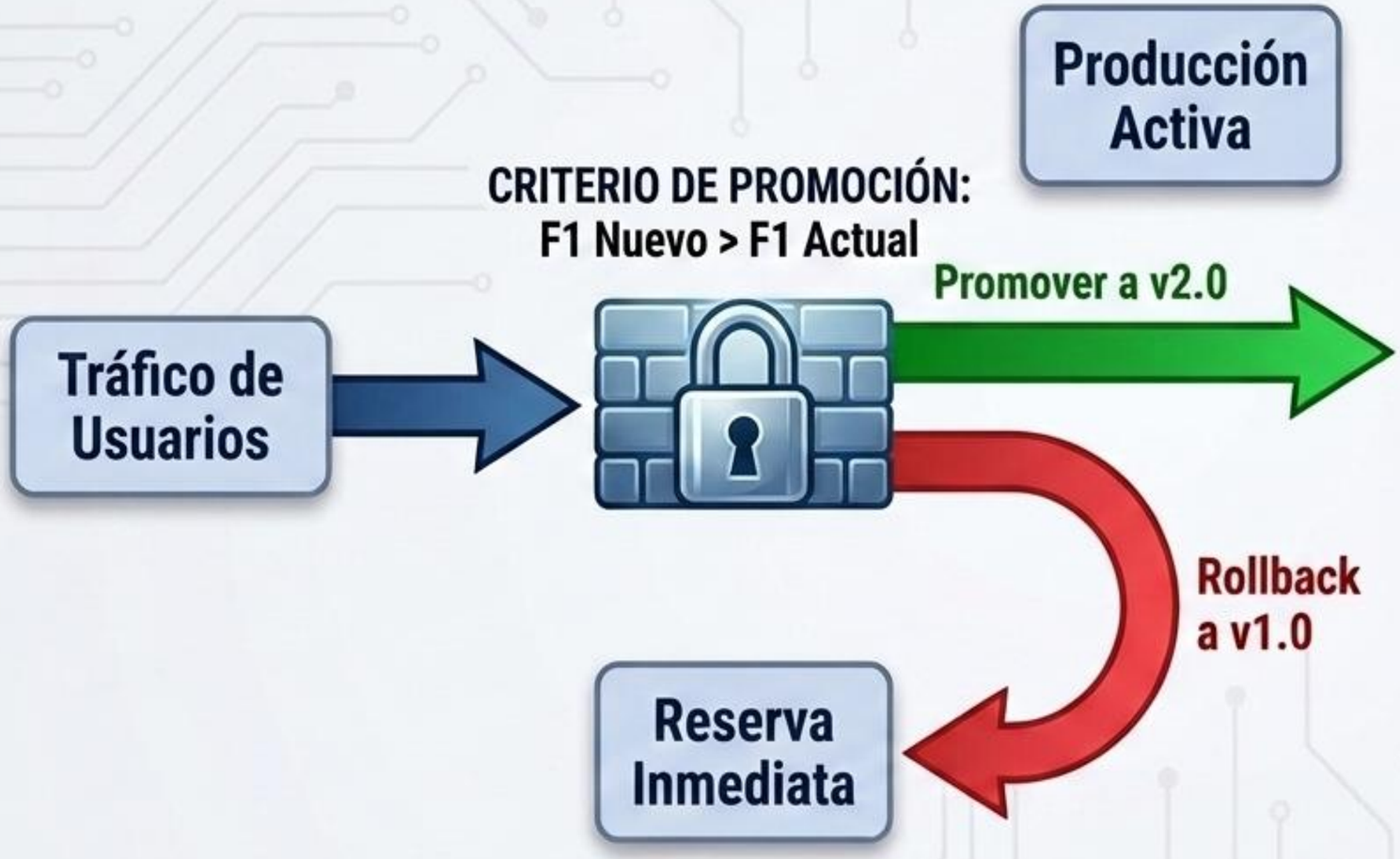


- Prohibido el uso de nombres amateur como `modelo_final_definitivo.keras`

- Un archivo binario sin metadatos adjuntos es inauditable y peligroso para la empresa.

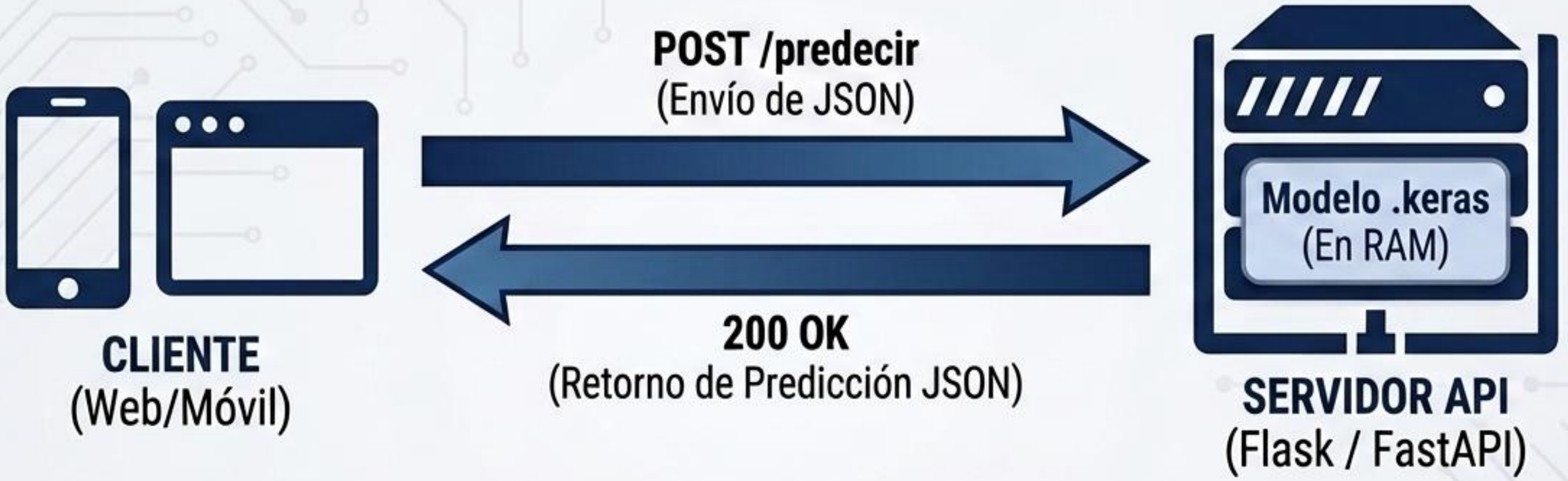
- Si en 6 meses el sistema se degrada, la ficha técnica revela las condiciones de entrenamiento originales.

Promoción matemática y protocolos de reversión (Rollback)



- **Promoción objetiva**: Una nueva versión solo reemplaza a la antigua si supera una métrica clave predefinida. No intervienen opiniones.
- **Shadow Mode**: Probar el modelo v2 con datos reales en silencio antes de darle el control total.
- **Rollback inmediato**: La versión anterior (v1) debe permanecer viva en la memoria para restaurar el servicio en segundos si la v2 colapsa.

La API REST aísla la carga matemática de la interfaz



- La API convierte el modelo en un microservicio agnóstico al lenguaje (consumible desde Java, JS, etc.).
- El cliente solo transmite datos; el servidor concentra toda la carga de cálculo matemático.
- Estandariza la comunicación a través del protocolo universal HTTP.

Flask: Regla de oro de la inicialización en memoria

```python
from flask import Flask, request, jsonify
import keras
app = Flask(__name__)

# CARGA GLOBAL ÚNICA EN MEMORIA RAM
modelo = keras.saving.load_model("modelo_v1.keras")

@app.route('/predecir', methods=['POST'])
def predecir():
    datos = request.json['datos']
    pred = modelo.predict([datos])
    return jsonify({"prediccion": float(pred[0][0])})
```

- **Flask** es maduro, directo y altamente configurable.
- **Error Crítico Evitado**: El archivo .keras NUNCA debe leerse dentro del endpoint /predecir.
- La lectura reiterada en disco duro colapsará el servidor bajo solicitudes concurrentes.
- El modelo debe esperar estático en la memoria RAM.

FastAPI: Tipado estricto y validación automática

```python
from fastapi import FastAPI
from pydantic import BaseModel
import keras

app = FastAPI()
modelo = keras.saving.load_model("modelo_v1.keras")

class Entrada(BaseModel):
    datos: list[float]  # VALIDACIÓN AUTOMÁTICA

@app.post('/predecir')
def predecir(req: Entrada):
    pred = modelo.predict([req.datos])
    return {"prediccion": float(pred[0][0])}
```

Escudo protector

- Emplea **Pydantic** para forzar la estructura exacta del JSON esperado.
- Si el cliente envía texto en vez de números, rechaza la solicitud de inmediato (Error 422 Automático).
- Protege a la red neuronal de estrellarse internamente (Error 500 evitado).
- Soporte nativo y superior para concurrencia asíncrona.

Decisión arquitectónica: Elegir el framework adecuado

Capacidad del Sistema	Flask	FastAPI
Madurez y Ecosistema	Muy alta (Estándar clásico)	Crecimiento masivo y moderno
Validación de Entradas	Manual (Código defensivo del dev)	Automática e integrada (Pydantic)
Documentación API	Configuración manual	Automática e interactiva (/docs)
Rendimiento Web	Sólido para cargas tradicionales	Superior (Diseñado para asincronía)

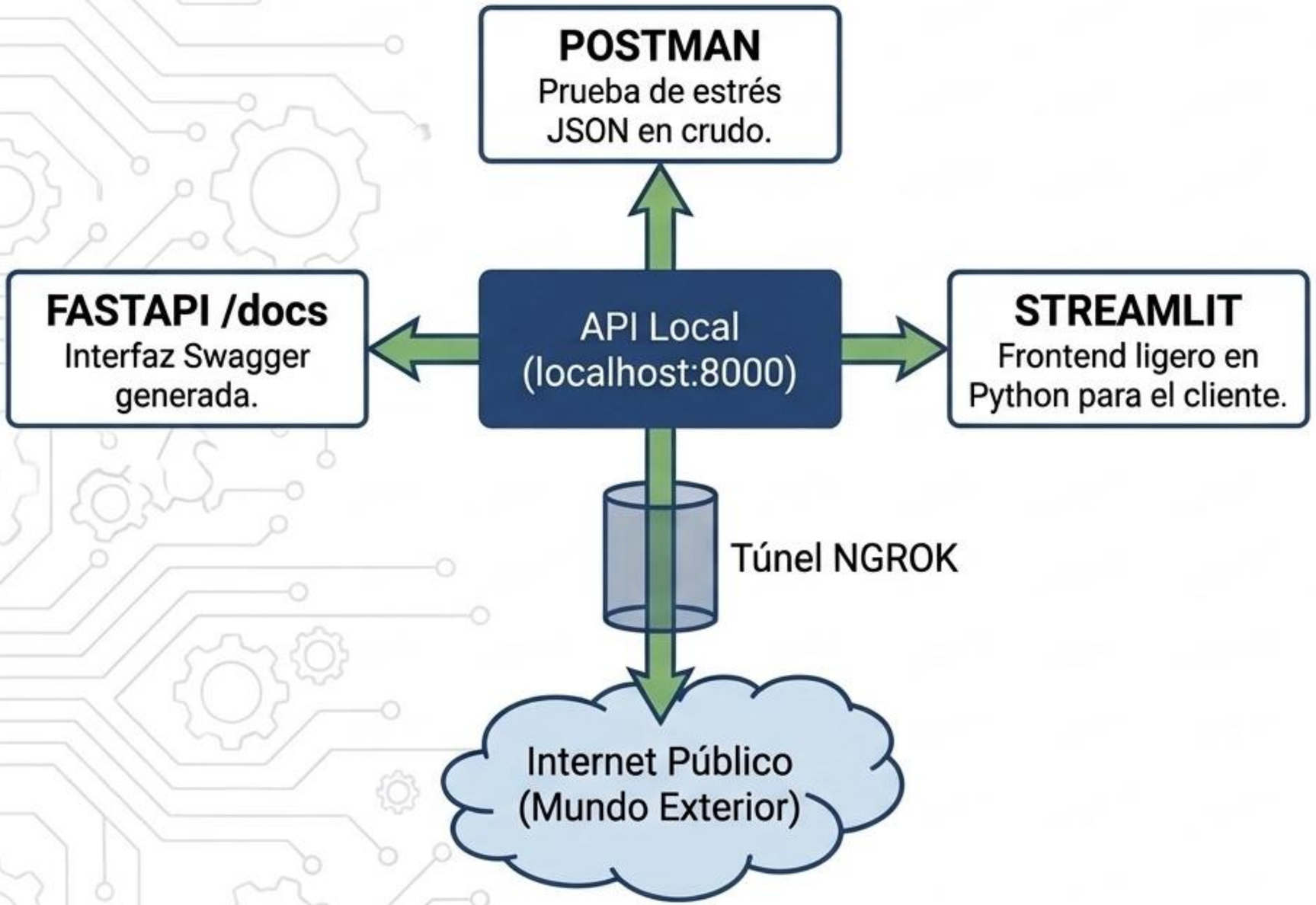
Punto en Común

Ambos resuelven excelentemente el mismo problema de exposición HTTP.

Factores de Decisión

La elección depende del contexto de ingeniería, legado de código de la empresa y escala de concurrencia requerida.

Interfaces de prueba y túneles de exposición



- **Postman:** Jamás integrar una API sin probar antes los endpoints crudos.

- **Streamlit:** Permite programar paneles interactivos que consumen nuestra API en minutos.

- **Ngrok:** Expone el entorno localhost a internet para auditorías remotas sin requerir servidores corporativos.

Síntesis operativa: Los 5 mandamientos del despliegue

Errores Fatales (X)	Estándar de Ingeniería (✓)
✗ Confiar ciegamente en la exactitud (Accuracy).	✓ Analizar Matrices, Recall y F1-Score.
✗ Filtrar datos del futuro en el entrenamiento.	✓ Aislar entornos y auditar variables temporales.
✗ Guardar los pesos sin la arquitectura subyacente.	✓ Versionar formatos nativos emparejados a metadatos.
✗ Cargar el archivo .keras con cada petición HTTP.	✓ Inicializar modelos estáticos en la memoria RAM global.
✗ Desplegar APIs sin validación de tipos de detos.	✓ Proteger endpoints con tipado estricto (Pydantic).

Laboratorio 06: De la teoría a la cinta transportadora

Preguntas de Diseño de Sistemas:
- ¿Qué pasa con las finanzas del hospital si el Recall de la IA falla?
- ¿Por qué acoplar el modelo algorítmico directamente al código móvil del cliente es un error arquitectónico?

Nivel Básico:
Evaluar modelo histórico (Matriz, F1).

Nivel Intermedio:
Construir la API local y probar con Postman.

Nivel Avanzado:
Frontend Streamlit y exposición web Ngrok.

- **Objetivo:** Someter los modelos de las semanas 4/5 al ciclo de vida completo.
- Su laptop personal se convertirá hoy en un servidor público de inteligencia artificial.